동적 포트폴리오

동적 프로그래밍을 활용한 동적 포트폴리오 구축

1. 문제 정의

각 시점 t에서 자산 배분 결정을 내릴 때, 포트폴리오의 총 수익을 극대화하거나 위험을 최소화하는 것을 목표로 합니다. 자산 배분 벡터를 x_t, 기대 수익을 R_t, 포트폴리오 가치 V_t로 정의합니다.

2. 벨만 방정식 설정

벨만 방정식은 다음과 같이 설정됩니다.

$$V_t = \max_{x_t} \{E[R_t(x_t)] + \beta V_{t+1}\}$$

여기서 β는 할인율을 나타내며, $E[R_t(x_t)]$는 x_t라는 자산 배분을 했을 때의 기대 수익입니다.

동적 프로그래밍을 활용한 동적 포트폴리오 구축

3. 상태와 결정 변수 정의

- **상태 변수**: S_t (시점 t에서의 포트폴리오 상태, 예: 현재 자산 가격, 시장 상황 등)
- **결정 변수**: x_t (자산 배분 벡터)

4. 동적 프로그래밍 구현 절차

1. **기댓값 계산**: 각 시점 t에서 자산 배분에 따른 수익 $R_t(x_t)$의 기댓값을 계산합니다.
2. **벨만 방정식 해결**: 미래 시점의 기대 가치를 고려해 현재 시점의 최적 배분 x_t를 찾습니다.
3. **재귀 최적화**: 최종 시점에서부터 시작해 역으로 현재 시점까지 최적의 결정을 계산합니다.

포트폴리오의 기대 수익률과 분산

파이썬 코드

```
import numpy as np

# 상태 및 결정 변수 설정
n_assets = 3
T = 10  # 투자 기간
gamma = 0.9  # 할인율

# 초기 상태: 각 자산의 수익률 예측 (임의 예시)
expected_returns = np.random.normal(0.05, 0.1, (T, n_assets))
cov_matrix = np.array([np.eye(n_assets) * 0.02 for _ in range(T)])
```

포트폴리오의 기대 수익률과 분산

```
# 벨만 방정식 함수 정의
def bellman_equation(t, V_next, expected_returns, cov_matrix, gamma):
    optimal_x = None
    max_value = -np.inf

    # 다양한 배분 비율에 대해 기대 가치 계산
    for _ in range(100):  # 배분 비율의 샘플링
        x_t = np.random.dirichlet(np.ones(n_assets))  # 임의 배분
        R_t = np.dot(expected_returns[t], x_t) - 0.5 * gamma * np.dot(x_t,
np.dot(cov_matrix[t], x_t))
        V_t = R_t + gamma * V_next
        if V_t > max_value:
            max_value = V_t
            optimal_x = x_t

    return max_value, optimal_x
```

포트폴리오의 기대 수익률과 분산

```
# 최적 포트폴리오 경로 계산
V = np.zeros(T + 1)
optimal_allocation = [None] * T

for t in reversed(range(T)):
    V[t], optimal_allocation[t] = bellman_equation(t, V[t + 1], expected_returns, cov_matrix, gamma)

print("최적 자산 배분 경로:", optimal_allocation)
```

동적 프로그래밍을 활용한 동적 포트폴리오 구축[수식]

1. 최적화 문제 정의

투자자가 T 기간 동안의 포트폴리오 가치를 극대화하기 위해 자산 배분을 최적화하고자 합니다.

최적화 문제:

$$\max_{\{x_t\}_{t=0}^{T}} E\left[\sum_{t=0}^{T} \beta^t R_t(x_t)\right]$$

여기서,

- x_t는 시점 t에서의 자산 배분 벡터로, 각 자산에 할당된 비율을 나타냅니다.
- $R_t(x_t)$는 자산 배분 x_t에 따른 시점 t의 기대 수익입니다.
- β는 할인율로, 미래 수익의 현재 가치를 반영합니다.

동적 프로그래밍을 활용한 동적 포트폴리오 구축[수식]

2. 벨만 방정식

벨만 방정식을 통해 현재 시점의 최적 결정을 미래 기대 가치에 의존하여 정의합니다.

벨만 방정식:

$$V_t(S_t) = \max_{x_t} \{E[R_t(x_t)|S_t] + \beta E[V_{t+1}(S_{t+1})|S_t, x_t]\}$$

여기서,

- $V_t(S_t)$는 시점 t에서의 최적 포트폴리오 가치입니다.
- S_t는 시점 t에서의 상태 벡터로, 시장 상태와 포트폴리오 구성에 관한 정보를 포함합니다.
- $E[R_t(x_t)|S_t]$는 상태 S_t 하에서의 자산 배분 x_t에 따른 기대 수익입니다.
- $E[V_{t+1}(S_{t+1})|S_t, x_t]$는 상태 S_t와 배분 x_t에 따른 다음 시점의 최적 가치의 기대값입니다.

동적 프로그래밍을 활용한 동적 포트폴리오 구축[수식]

3. 기댓값 계산

각 시점에서 자산 배분에 따른 기대 수익을 계산합니다. 예를 들어, 다음과 같이 정의할 수 있습니다:

$$E[R_t(x_t)] = x_t' \mu_t - \frac{\gamma}{2} x_t' \Sigma_t x_t$$

여기서,

- μ_t는 시점 t에서 각 자산의 기대 수익률 벡터입니다.
- Σ_t는 자산 간 공분산 행렬로, 리스크를 반영합니다.
- γ는 투자자의 위험회피 계수입니다.

이 수식에서 $x_t' \mu_t$는 자산 배분 x_t에 따른 기대 수익이며, $\frac{\gamma}{2} x_t' \Sigma_t x_t$는 리스크를 고려한 조정 항입니다.

동적 프로그래밍을 활용한 동적 포트폴리오 구축[수식]

4. 최적 자산 배분 결정

시점 t에서의 최적 자산 배분 x_t^*는 벨만 방정식을 통해 결정됩니다.

최적 자산 배분:

$$x_t^* = \arg\max_{x_t} \{E[R_t(x_t)] + \beta V_{t+1}(S_{t+1})\}$$

이때, 이 x_t^*는 기대 수익과 미래 가치의 기대값을 최대화하는 자산 배분입니다.

5. 재귀적 계산

동적 프로그래밍에서 재귀적으로 $t = T$부터 $t = 0$까지 최적의 자산 배분과 포트폴리오 가치를 계산합니다. 최종 시점에서의 가치 $V_T(S_T)$는 종료 조건으로 주어지며, 이를 기반으로 역방향으로 최적의 경로를 찾아갑니다.

재귀적 벨만 방정식:

$$V_t(S_t) = E[R_t(x_t^*)|S_t] + \beta V_{t+1}(S_{t+1})$$

이 재귀 방정식을 통해 각 시점에서의 최적 자산 배분 x_t^*와 포트폴리오 가치 V_t를 계산할 수 있습니다.

강화학습 풀이

강화학습을 적용하는 단계

1. **상태 정의**: 포트폴리오 관리 상황에서 상태 S_t는 자산의 현재 가격, 과거 수익률, 경제 지표 등 포트폴리오의 상태를 나타냅니다.
2. **행동 정의**: 에이전트가 각 시점에서 선택할 수 있는 행동 a_t는 자산 배분 벡터입니다. 즉, 어떤 자산에 얼마나 배분할지를 결정합니다.
3. **보상 함수 정의**: 보상 R_t는 자산 배분 a_t에 따른 포트폴리오의 수익으로 설정됩니다. 예를 들어, 포트폴리오의 수익률 또는 리스크-조정 수익률을 보상으로 설정할 수 있습니다. 특정 목표를 달성하지 못할 경우 패널티를 추가할 수도 있습니다.
4. **정책 학습**: 에이전트는 상태와 행동의 조합을 통해 최적의 기대 보상을 얻기 위한 정책 $\pi(a|S)$를 학습합니다. 이 정책은 현재 상태에서의 최적 행동을 결정하는 함수로, Q-러닝이나 DDPG(Deep Deterministic Policy Gradient) 등의 알고리즘을 사용할 수 있습니다.

강화학습 풀이

5. **벨만 최적 방정식 기반 강화학습**: 벨만 최적 방정식을 강화학습에서 다음과 같이 사용할 수 있습니다:

$$Q(S_t, a_t) = R_t + \gamma \max_{a_{t+1}} Q(S_{t+1}, a_{t+1})$$

   여기서 $Q(S_t, a_t)$는 상태-행동 값 함수(Q-function)로, 현재 상태 S_t에서 행동 a_t를 선택했을 때의 기대 보상을 나타냅니다.

6. **모델 학습 및 시뮬레이션**: 다양한 마켓 시나리오에서 포트폴리오 관리 시뮬레이션을 반복하면서 에이전트가 최적의 자산 배분 정책을 학습하도록 합니다. 시뮬레이션을 통해 에이전트가 다양한 상태에서 최적의 행동을 학습하고, 이를 실제 시장 상황에 적용할 수 있게 됩니다.

강화학습 알고리즘 예시

포트폴리오 최적화에는 여러 강화학습 알고리즘을 사용할 수 있습니다.

- **DQN (Deep Q-Network)**: 이산적인 자산 배분 비율을 설정하여 자산 배분 문제를 해결할 수 있습니다.
- **DDPG (Deep Deterministic Policy Gradient)**: 연속적인 자산 배분이 필요한 경우 사용하며, 포트폴리오의 자산 배분 비율을 연속적으로 최적화합니다.
- **PPO (Proximal Policy Optimization)**: 안정적이고 신뢰할 수 있는 정책을 학습하기 위한 강화학습 알고리즘으로, 시장의 불확실성을 잘 다룰 수 있습니다.

이렇게 강화학습을 적용하면 에이전트가 과거 데이터 또는 시뮬레이션을 통해 최적의 자산 배분 전략을 스스로 학습하게 되어, 동적 포트폴리오 관리 문제를 효과적으로 해결할 수 있습니다.

동적 프로그래밍과 강화학습의 비교

1. 모델의 가정 여부

- **Dynamic Programming**: DP는 환경의 상태 전이 확률과 보상 함수를 미리 알고 있어야 적용할 수 있는 방법입니다. 따라서 포트폴리오의 미래 수익이나 시장 변동에 대한 확률 모델이 주어져 있는 경우 적합합니다. 예를 들어, 시장의 수익률이나 변동성을 특정한 모델(예: 확률적 프로세스)로 가정할 때 DP를 통해 최적화할 수 있습니다.
- **Reinforcement Learning**: RL은 환경의 상태 전이 확률이나 보상 함수를 몰라도 최적 정책을 학습할 수 있습니다. 과거의 데이터를 기반으로 에이전트가 시장의 불확실성 속에서 점진적으로 학습하므로, 시장의 동학을 정확히 모델링하기 어려운 경우에 더 유용합니다. RL은 시뮬레이션 환경을 통해 에이전트가 행동을 반복적으로 시도하면서 최적의 자산 배분을 학습하게 합니다.

동적 프로그래밍과 강화학습의 비교

2. 목표 설정 방식

- **DP**: DP는 벨만 방정식을 기반으로 최적의 결정 방식을 재귀적으로 계산해 나가며, 수식적으로 명확히 정의된 최적화 문제를 해결합니다. 최종 목표는 벨만 방정식에서 정의된 기대 가치를 최대화하는 것이며, 이를 통해 포트폴리오 가치를 최적화하는 정책을 도출합니다.
- **RL**: RL에서는 에이전트가 목표를 달성하기 위한 최적의 정책을 보상을 통해 학습합니다. 보상은 에이전트가 포트폴리오에서 기대 수익을 높이는 행동을 취할 때 긍정적으로 피드백되며, 반대로 리스크가 높아지는 방향의 행동에 대해 페널티가 주어질 수도 있습니다. 학습 과정에서 정책이 최적화되므로 DP처럼 명확히 수식을 사용해 최적 정책을 정의하지 않습니다.

동적 프로그래밍과 강화학습의 비교

3. 환경의 불확실성 대응

- **DP**: DP는 정해진 확률 모델을 통해 불확실성을 다룹니다. 하지만 환경이 동적이거나 예측하기 어려운 경우, DP를 활용하기 어려울 수 있습니다. 따라서 DP는 고정된 확률적 모델에서만 효과적으로 작동하며, 예기치 않은 변화에 대응하는 유연성은 낮습니다.
- **RL**: RL은 실시간으로 에이전트가 환경에 적응하며, 예상치 못한 상황에서도 최적의 행동을 찾아나갈 수 있습니다. 특히 DDPG, PPO 같은 알고리즘을 사용하면 연속적인 포트폴리오 비율을 실시간으로 조정할 수 있어 시장 상황 변화에 더 유연하게 대처할 수 있습니다.

동적 프로그래밍과 강화학습의 비교

4. 연산 복잡도 및 계산 효율성

- **DP**: DP는 벨만 방정식을 단계별로 재귀적으로 해결하기 때문에, 상태 공간이 크면 계산량이 급격히 증가합니다. 상태와 행동의 가능한 조합이 많을 경우 비효율적일 수 있으며, 고차원의 연속 상태 공간에 대해 확장성이 떨어집니다.
- **RL**: RL은 대규모 상태 공간에서도 학습을 통해 효율적인 정책을 찾아낼 수 있습니다. 특히 심층 신경망을 이용한 RL(DQN, DDPG 등)은 복잡한 고차원 상태 공간에서도 유효한 정책을 학습할 수 있습니다. 하지만 이 과정에서 충분한 학습 시간과 데이터가 필요하므로, 훈련 시간이 길어질 수 있습니다.

동적 프로그래밍과 강화학습의 비교

요약

- **Dynamic Programming**: 명확히 정의된 모델과 상태 전이 확률이 주어져 있는 경우 최적화에 유리하며, 계산 효율성 문제로 인해 고차원의 연속 공간에서 사용하기 어렵습니다.
- **Reinforcement Learning**: 불확실성이 크거나 확률 모델을 알기 어려운 상황에서 최적화에 유리하며, 유연성과 확장성이 높아 복잡한 시장 환경에서 실시간 포트폴리오 최적화를 수행하는 데 적합합니다.

모델 기반 강화학습과 동적 프로그래밍

Model-Based 강화학습의 특징

1. **환경 모델 사용**: Model-based RL은 에이전트가 환경 모델(예: 상태 전이 확률, 보상 함수)을 추정하거나 학습하도록 합니다. 이를 통해 에이전트가 직접 시도해 보지 않은 행동의 결과를 예측할 수 있어 학습 속도를 높일 수 있습니다.
2. **시뮬레이션을 통한 행동 평가**: 에이전트는 모델을 활용해 가상의 시뮬레이션을 수행하여 각 행동의 결과를 시뮬레이션하고, 이를 통해 최적의 행동을 선택합니다. 이를 통해 실제 환경과의 상호작용을 줄이며 효율적인 학습이 가능합니다.
3. **계산 효율성**: 환경 모델을 통해 미래의 상태를 예측하므로, 시뮬레이션 내에서 많은 상태-행동 쌍을 탐색할 수 있습니다. 이는 model-free RL보다 학습 효율이 높고 데이터 효율이 뛰어납니다.

모델 기반 강화학습과 동적 프로그래밍

Dynamic Programming (DP)와 Model-Based RL의 차이점

1. **모델 사용 여부와 방식**
   - **Dynamic Programming**: DP는 기본적으로 환경의 모델이 완전히 주어졌을 때 사용합니다. 즉, 상태 전이 확률과 보상 함수가 명확히 정의되어 있고, 이를 통해 최적의 정책을 수식적으로 계산합니다. 이 과정에서 학습 과정은 필요하지 않습니다.
   - **Model-Based RL**: 모델이 주어지지 않았거나 불완전한 경우, 에이전트가 학습 과정에서 환경 모델을 추정하거나 학습합니다. 모델이 확실하게 정의되어 있지 않은 경우에도 경험을 통해 추정된 모델을 사용해 최적의 정책을 학습할 수 있습니다.
2. **미지의 환경에 대한 적용 가능성**
   - **DP**: 모델이 명확히 정의된 환경에서만 적용이 가능하므로, 실제 응용 환경에서 사용하기 어려울 수 있습니다.
   - **Model-Based RL**: 에이전트가 경험을 통해 환경 모델을 학습할 수 있어 미지의 환경에서도 적용 가능성이 높습니다. 실제 환경과의 상호작용을 통해 모델을 갱신하고, 유동적이거나 복잡한 환경에서도 학습할 수 있습니다.

모델 기반 강화학습과 동적 프로그래밍

3. **학습 방식**
    - **DP**: 상태 전이 모델이 완전하므로 학습 과정이 없이 벨만 방정식을 통해 최적 정책을 바로 계산할 수 있습니다.
    - **Model-Based RL**: 모델이 없는 상태에서 시작해 경험을 통해 모델을 학습하고, 이를 기반으로 정책을 최적화합니다. Model-Free RL과 달리 모델을 학습하는 과정이 추가되지만, 정책 학습의 효율성을 높일 수 있습니다.
4. **탐색 및 최적화**
    - **DP**: 각 시점의 상태 전이를 모두 알고 있어 탐색 없이 즉시 최적화를 수행합니다. 이는 상태 공간이 크면 비효율적일 수 있습니다.
    - **Model-Based RL**: 환경 모델을 통해 가상 시뮬레이션을 반복 수행하여 최적의 행동을 찾습니다. 이를 통해 데이터 효율성을 높이지만, 모델을 학습하는 데 있어 탐색이 필요합니다.

모델 기반 강화학습과 동적 프로그래밍

요약

- **Dynamic Programming (DP)**: 완전한 환경 모델이 있을 때 벨만 방정식을 통해 최적 정책을 계산하는 방식입니다. 학습 과정 없이 최적화를 수행하며, 복잡한 상태 공간에서는 비효율적일 수 있습니다.
- **Model-Based RL**: 환경 모델이 불확실하거나 없을 때 에이전트가 학습 과정을 통해 환경 모델을 구축하고, 이를 통해 최적의 행동을 예측하는 방식입니다. DP보다 유연성이 높으며, 미지의 환경에서도 학습과 탐색을 통해 최적 정책을 찾는 데 유리합니다.

강화학습 개선

강화학습에서 목적 함수에 **엔트로피(entropy)**를 추가하는 접근법은 정책의 **탐색성(exploration)**을 높이고, **정책의 확률 분포를 더욱 부드럽게 만드**는 효과가 있습니다. 엔트로피 항을 통해 정책이 특정 행동에 과도하게 집중되지 않도록 하고, 탐색을 장려하여 더 안정적이고 일반화된 학습을 가능하게 합니다.

1. 엔트로피를 활용한 목적 함수

엔트로피를 추가한 목적 함수는 다음과 같이 정의됩니다:

$$J(\theta) = E\left[\sum_{t-0}^{T} \gamma^t r_t\right] + \alpha \sum_{t-0}^{T} H(\pi(\cdot|s_t))$$

여기서:

- $J(\theta)$는 최적화하려는 목적 함수입니다.
- $H(\pi(\cdot|s_t)) = -\sum_a \pi(a|s_t) \log \pi(a|s_t)$는 상태 s_t에서 정책 π의 엔트로피입니다.
- α는 엔트로피 항의 가중치를 조절하는 하이퍼파라미터로, 탐색과 정책의 확률 분포의 균일성을 제어합니다.

이 엔트로피 항을 추가하면 에이전트가 특정 행동에 대해 확신을 가지기 전까지 여러 행동을 더 폭넓게 탐색하게 되어, 과도한 수렴을 방지하고 정책의 다형성을 유지하는 데 도움이 됩니다.

강화학습 개선

2. 엔트로피 정규화의 효과

- **탐색성 강화**: 엔트로피 항이 포함되면 정책이 다양한 행동을 선택하게 되어, 새로운 정보와 상황에 대해 더 많이 탐색할 수 있습니다. 특히 초기 학습 단계에서 과도하게 특정 행동을 고수하지 않도록 유도합니다.
- **소프트한 의사결정**: 특정 행동을 강하게 선택하지 않도록 함으로써, 정책이 여러 행동에 대해 확률적으로 분포되도록 유도합니다. 이는 주로 불확실한 상황에서 최적의 행동을 찾지 못할 때 유용하게 작용합니다.
- **정책의 안정성**: 엔트로피 정규화는 학습의 안정성을 높이고, 최종 정책이 지나치게 편향되지 않도록 합니다. 결과적으로, 엔트로피를 추가한 정책은 다양한 상태에서 더 일관된 성능을 보일 수 있습니다.

강화학습 개선

3. 주요 알고리즘

- **Soft Actor-Critic (SAC)**: 엔트로피 정규화가 대표적으로 사용되는 강화학습 알고리즘으로, 목적 함수에 엔트로피 항을 추가하여 정책이 '소프트'하게 행동하도록 합니다. SAC의 경우, 목표는 총 보상을 극대화하는 동시에 엔트로피 항을 최대화하여 다양한 행동 선택을 장려하는 것입니다.
- **Proximal Policy Optimization (PPO)**: PPO에서도 엔트로피 보너스를 추가하여 탐색성을 높이는 경우가 있으며, 이를 통해 특정 행동에 대한 지나친 집중을 피하도록 합니다.

강화학습 개선

4. 하이퍼파라미터 α의 선택

엔트로피 항의 가중치인 α의 크기는 정책의 탐색 정도에 큰 영향을 미칩니다.

- α가 크면 엔트로피가 높아져 더 많은 탐색이 이루어지며, 정책이 느리게 수렴할 수 있습니다.
- α가 작으면 정책이 더 빨리 수렴하나, 탐색이 제한되어 최적의 정책을 찾기 어려울 수 있습니다.

엔트로피 보너스를 활용한 강화학습은 특히 불확실한 환경에서 정책의 탐색성을 높이고, 안정적이면서도 다양한 행동 선택을 가능하게 하는 강력한 방법입니다.

SAC(Soft Actor Critic)

SAC의 주요 아이디어

1. **엔트로피 정규화**: SAC는 엔트로피를 통해 정책의 **탐색성**을 강화하고 **소프트한 의사결정**을 장려합니다. 엔트로피 항을 추가함으로써, 에이전트가 특정 행동에 과도하게 집중하는 것을 방지하고 다양한 행동을 시도하게 합니다. 이를 통해 에이전트는 더 폭넓은 상태-행동 공간을 탐색하여 학습의 초기 단계에서 더 많은 데이터를 수집하게 됩니다.

2. **Off-policy 학습**: SAC는 **off-policy** 알고리즘이므로, 이전에 수집된 데이터를 재사용하여 학습합니다. 이를 통해 데이터 효율성을 높이며, 효율적으로 메모리 버퍼(replay buffer)를 사용하여 더 나은 샘플 효율성을 얻을 수 있습니다.

3. **두 개의 Q 함수와 정책 네트워크 사용**: SAC는 **클리핑 없이 학습 안정성을 향상**시키기 위해 두 개의 Q 함수 Q_θ^1와 Q_θ^2를 사용합니다. 두 Q 함수 중 최소값을 사용하여 **과대평가를 방지**하며, 신뢰할 수 있는 가치 추정치를 제공합니다. 또한, **정책 네트워크와 가치 함수**를 분리하여 배우고, 정책을 위한 두 개의 신경망과 평가를 위한 두 개의 Q 함수를 따로 학습합니다.

4. **자동 온도 조정 (Automatic Temperature Adjustment)**: SAC는 학습 중 **온도 파라미터 α**를 자동으로 조정하여 탐색성과 안정성을 제어합니다. 온도 파라미터는 엔트로피의 가중치를 조절하며, SAC는 **타겟 엔트로피(target entropy)**를 기준으로 α를 동적으로 학습합니다. 타겟 엔트로피가 높을수록 탐색성이 증가하고, 낮을수록 더 안정된 정책으로 수렴합니다.

SAC(Soft Actor Critic)

SAC의 목적 함수

SAC는 보상뿐 아니라 **엔트로피 항을 포함하는 목적 함수**를 최적화합니다. SAC의 목적 함수는 다음과 같이 정의됩니다:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi} \left[r(s_t, a_t) + \alpha H(\pi(\cdot|s_t)) \right]$$

여기서,

- $r(s_t, a_t)$는 시점 t에서의 보상입니다.
- $H(\pi(\cdot|s_t))$는 정책 π의 엔트로피로, $H(\pi(\cdot|s_t)) = -\sum_a \pi(a|s_t) \log \pi(a|s_t)$입니다.
- α는 엔트로피 항의 가중치로, 탐색성과 안정성을 조절하는 역할을 합니다.

SAC는 **정책 네트워크 π_ϕ**와 **Q 함수 Q_θ^1와 Q_θ^2**를 학습합니다. 학습 과정은 다음과 같은 단계로 이루어집니다.

SAC(Soft Actor Critic)

SAC의 학습 과정

1. **Q 함수 업데이트:**
   - 두 Q 함수 Q_θ^1와 Q_θ^2의 손실 함수는 다음과 같이 정의됩니다.

$$J_Q(\theta) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(Q_\theta(s,a) - \left(r + \gamma\left(\min_{i-1,2} Q_{\theta'}^i(s',a') - \alpha \log \pi_\phi(a'|s')\right)\right)\right)^2\right]$$

   - 여기서 a'는 다음 상태 s'에서의 행동을 나타내며, θ'는 타겟 네트워크의 파라미터입니다.
   - Q 함수는 현재 상태-행동 쌍에 대해 가치 함수를 추정하며, 두 Q 값 중 최소값을 사용해 과대평가를 방지합니다.

SAC(Soft Actor Critic)

2. **정책 네트워크 업데이트**:
   - 정책 네트워크 π_ϕ는 엔트로피를 포함한 손실 함수를 통해 업데이트됩니다.

$$J_\pi(\phi) = \mathbb{E}_{s\sim D}\left[\alpha \log \pi_\phi(a|s) - \min_{i-1,2} Q_\theta^i(s,a)\right]$$

   - 여기서 정책은 상태 s에 대해 높은 엔트로피를 유지하면서 보상 최대화를 동시에 목표로 합니다. 엔트로피가 포함된 정책 손실을 최소화함으로써 안정적이면서도 탐색성이 높은 정책을 학습합니다.

3. **온도 파라미터 업데이트**:
   - 타겟 엔트로피에 따라 자동으로 온도 파라미터 α를 학습하여 탐색성 조절을 자동화합니다.

$$J(\alpha) = \mathbb{E}_{a\sim\pi}\left[-\alpha\left(\log \pi_\phi(a|s) + \mathcal{H}_{\text{target}}\right)\right]$$

   - 이 과정에서 엔트로피가 타겟 엔트로피에 가까워지도록 α를 동적으로 조절합니다.

SAC(Soft Actor Critic)

SAC의 장점

- **효율적인 탐색과 안정성**: 엔트로피 보너스를 통해 다양한 행동을 탐색하면서도 수렴이 안정적입니다.
- **자동 탐색성 조절**: 타겟 엔트로피를 통해 자동으로 탐색성과 안정성을 조절하여, 환경에 적응할 수 있습니다.
- **Off-policy 학습의 데이터 효율성**: 재사용 가능한 샘플을 통해 학습하여 데이터 효율성을 높입니다.

SAC는 이러한 특징 덕분에 연속적이고 불확실한 환경에서도 강건한 성능을 보이는 강화학습 알고리즘입니다.